

Q2 Frequency Greater than k

-1

① psum()

```
int arr = {1, 1, 2, 2, 2, 3}
```

```
int k = 2;
```

```
for(int i = 0; i < arr.length; i++)
```

```
int check = arr[i];
```

```
for(int j = 0; j < arr.length; j++)
```

```
if (check == arr[j])
```

```
count++;
```

```
}
```

```
}
```

```
if (count > k)
```

```
S.O.P(arr[i]);
```

```
}
```

```
}
```

```
}
```

② psum()

```
int arr = {1, 1, 2, 2, 2, 3};
```

```
int freq = new int[arr.length + 1];
```

```
int k = 2;
```

$$f_n(t) = 0.12 \exp(-0.12t)$$

S.O.P ("Freq (i,j))

9. (3) psu(m) }

HashMap <Integer, Integer> has new HashMap <> ()

$$\text{hm_put}(\text{arr}[i], \text{get}(\text{O}, \text{Default}(\text{arr}[i], 0) + 1));$$
$$\{ \text{if } \text{length}(\text{key}) > k \}$$

① $\{p_{2n+1}\}$

$$\text{len}(s) \leq 0 : \text{return } s + ''$$
$$\text{Vol}(\pi_1) = 0 \text{ if } \text{rank}(\pi_1) \geq 1$$

A hand-drawn sketch on a lined background. It consists of two wavy, horizontal lines. The top line is a simple, slightly curved wavy line. The bottom line is a more complex, multi-lobed wavy line, resembling a stylized wave or a series of connected 'u' shapes.

2	3
---	---

2) word

$\text{len}(s) = 0$; is arr. length; it's }

freq [auct] 77


```

        for (int i = 0; i < arr.length; i++) {
            if (freq[i] == k) {
                S.O.P(freq[i]);
            }
        }
    }
}

```

③ sum()

```

int [] arr = {1, 1, 2, 2, 2, 3}
int k = 2;

```

```

HashMap<Integer, Integer> hm = new HashMap<>();

```

```

for (int i = 0; i < arr.length; i++) {

```

```

    hm.put(arr[i], hm.getOrDefault(arr[i], 0) + 1);
}

```

```

for (<Integer> key : hm.keySet()) {

```

```

    if (hm.get(key) == k) {

```

```

        S.O.P(key);
    }
}
}

```

```

}

```

Q4 Elements Appearing Only Once

① sum()

```

int [] arr = {1, 1, 2, 2, 2, 3}

```

```

for (int i = 0; i < arr.length; i++) {

```

```

    int check = arr[i];

```

```

    for (int j = 0; j < arr.length; j++) {

```

```

        if (check == arr[j]) {

```

```

            count++;
        }
    }
}

```

```

if (count == 1) {

```

```

    S.O.P(arr[i]);
}
}
}

```

② sum()

```

int [] arr = {1, 1, 2, 2, 2, 3}

```

```

int [] freq = new int[arr.length + 1];

```

```

for (int i = 0; i < arr.length; i++) {

```

```

    freq[arr[i]]++;
}
}

```

```

for(int i=0; i<arr.length; i++) {
    if(hmap.get(i) == 1) {
        S.O.P(hmap.get(i));
    }
}
}

```

③

```

psum() {
    int arr[] = {1, 1, 2, 2, 2, 3};
    HashMap<Integer, Integer> hmap = new HashMap<>();
    for(int i=0; i<arr.length; i++) {
        hmap.put(arr[i], hmap.getOrDefault(arr[i], 0) + 1);
    }
    for(<Integer> key : hmap.keySet()) {
        if(hmap.get(key) == 1) {
            S.O.P(key);
        }
    }
}
}

```

Q5. Remove Duplicates from array.

① psum() {

```

int[] arr = {1, 2, 2, 2, 3};
int[] newarr = new int[arr.length];

```

```

for(int i=0; i<arr.length; i++) {

```

```

    int check = arr[i];

```

```

    for(int j=0; j<arr.length; j++) {

```

```

        if(check == arr[j]) {
            count++;
        }
    }

```

```

}

```

```

if(count == 1) {

```

```

    count = 0;
    newarr[i] = arr[i];
}
}

```

```

} S.O.P(newarr);
}

```

② psum() {

```

int[] arr = {1, 2, 2, 2, 3};

```

```

int[] freq = new int[arr.length+1];

```

```

int[] newarr = new int[arr.length];

```

```
for(int i=0; i<arr.length; i++)  
{  
    freq[arr[i]]++;  
}
```

```
for(int i=0; i<arr.length; i++)  
{  
    if(freq[i] == 1){  
        newarr[i] = arr[i];  
    }  
}
```

③ print()

```
int[] arr = {1, 2, 2, 2, 3};  
int[] newarr = new int[arr.length];  
int count = 0;  
HashMap<Integer, Integer> hm = new HashMap<>();  
for(int i=0; i<arr.length; i++)  
{  
    hm.put(arr[i], getOrDefault(hm, arr[i], 0) + 1);  
}
```

```
for(int i=0; i<arr.length; i++)  
{  
    hm.put(arr[i], getOrDefault(hm, arr[i], 0) + 1);  
}
```

```
for(int i=0; i<arr.length; i++)  
{  
    if(hm.get(arr[i]) == 1){  
        newarr[count] = arr[i];  
        count++;  
    }  
}
```

Qc Most frequent character in String.

① print()

```
String str = "apple";  
int max = 0;  
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```

```
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```

```
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```

```
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```

```
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```

② print()

```
String str = "apple";  
int max = 0;  
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```

```
for(int i=0; i<str.length(); i++)  
{  
    char ch = str.charAt(i);  
    for(int j=0; j<str.length(); j++)  
    {  
        if(str.charAt(j) == ch){  
            count++;  
        }  
    }  
    if(count > max){  
        max = str.charAt(i);  
    }  
}
```



```

char check = str.charAt(i);
int count = 0;
for (int j = 0; j < str.length(); j++) {
    if (check == str.charAt(j)) {
        count++;
    }
}
if (count > max) {
    max = count;
}
}

```

② psum()

```

String str = "apple";
int max = 0;
char maxChar = ' ';
HashMap<Character, Integer> hm = new HashMap<>();
for (int i = 0; i < str.length(); i++) {
    hm.put(str.charAt(i), hm.getOrDefault(str.charAt(i), 0) + 1);
}
for (Character key : hm.keySet()) {
}

```

```

if (hm.get(key) > max) {
    max = hm.get(key);
    maxKey = key;
}
}

```

S.O.P(maxKey);

Q. Check if String is Pangram (a-z)

① psum()

```

String str = "the quick brown fox jumps over the lazy dog";
boolean found = false;
for (int i = 0; i < str.length(); i++) {
}

```

```

for (int j = 0; j < str.length(); j++) {
    if (str.charAt(i) == str.charAt(j)) {
        found = true;
        break;
    }
}
}

```

```

if (!found) {
    return false;
}
}

```

②

psum()

```

String str = "the quick brown fox jumps over the lazy dog";
boolean[] freq = new boolean[26];

```

```

for(int i=0; i<str.length(); i++)
    char ch = str.charAt(i);
    if(ch >= 'a' & ch <= 'z')
        freq[ch-'a'] = true;
}
}

```

```

for(boolean found : freq)
    if(!found)
        return false;
}
}

```

③

perm11

String str = "The quick brown fox jumps over the lazy dog";

HashMap<Character, Integer> hm = new HashMap<>();

```
for(int i=0; i<str.length(); i++)
```

```
    char ch = str.charAt(i);
```

```
    if(ch >= 'a' & ch <= 'z')
```

```
        hm.put(ch, hm.getOrDefault(ch, 0) + 1);
```

```
    }
```

```
}
```

```
if(hm.size() == 26)
```

```
    return true;
```

```
}
```

Q9. Find missing elements both has array

① perm03

```
int[] arr1 = {1, 2, 2, 2, 3, 7};
```

```
int[] arr2 = {1, 1, 2, 2, 3};
```

```
for(int i=0; i<arr1.length; i++)
```

```
    int check = arr1[i];
```

```
for(int j=0; j<arr2.length; j++)
```

```
    if(check == arr2[j])
```

```
        S.O.P("Match");
```

```
    } else {
```

```
        S.O.P(check);
```

```
    }
```

```
}
```

```
}
```

②

perm11

```
int[] arr1 = {1, 1, 2, 2, 2, 3, 7}
```

```
int[] arr2 = {1, 1, 1, 2, 2, 3, 7}
```